

Pipeline de Datos del Macroentorno Ecuatoriano

De archivos crudos de 9 fuentes públicas a un dashboard analítico en Power BI, siguiendo arquitectura medallón
Bronze → Silver → Gold.

6to ciclo

Practicum 2.2

Línea Datos

Un pipeline end-to-end, de dato crudo a decisión estratégica

El proyecto transforma archivos crudos de 9 fuentes públicas ecuatorianas (BCE, INEC, Supercias, MINEDUC) en un dashboard analítico de Power BI capaz de responder 3 preguntas estratégicas sobre la economía del Ecuador, siguiendo la arquitectura medallón Bronze → Silver → Gold.

9

fuentes públicas
de datos

14

tablas en
PostgreSQL 18

6

vistas Gold
para el dashboard

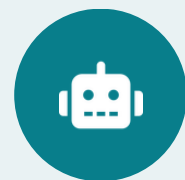
3

preguntas
estratégicas

Preguntas que responde el dashboard:

- P1 — ¿Cómo ha evolucionado la economía ecuatoriana en 20 años?
- P2 — ¿Cómo se comportan los sectores y provincias del país?
- P3 — ¿Qué relación existe entre bachilleres y empresas por provincia?

Arquitectura medallón: de la fuente al dashboard



Semana 5+: automatización RPA

Un RPA deposita archivos en `datos_macroentorno/`. `pipeline.py` detecta y procesa los archivos nuevos sin intervención manual.

9 archivos fuente, 4 organismos públicos



5

archivos

Banco Central del Ecuador

PIB, VAB, petróleo, riesgo país e IEE
— series de 1965 a 2026



2

archivos

INEC

Mercado laboral (ENEMDU) y Censo de Población 2022



2

archivos

Superintendencia de Compañías

Ranking empresarial y directorio



1

archivos

MINEDUC

Registros administrativos AMIE 2023-2024

Estructura del repositorio macroentorno_pipeline



pipeline.py

Orquestador que detecta y procesa archivos depositados por el RPA (semana 5)



transform/

4 scripts ETL — bce.py, inec.py, supercias.py, mineduc.py



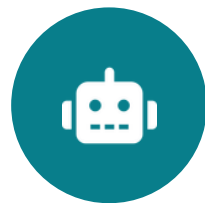
sql/

create_tables.sql (14 tablas) y gold_views.sql (6 vistas)



datos_crudos/

Capa Bronze, excluida de Git vía .gitignore



datos_macroentorno/

Carpeta de aterrizaje del RPA (semana 5+)



venv_datos/ • .env

Entorno virtual y credenciales, fuera del control de versiones

bce.py — 5 funciones, 5 tablas Silver

Objetivo: Procesar el PIB (Real y Nominal), Valor Agregado Bruto (VAB), Petróleo y Riesgo País.

Tablas Generadas: silver_pib_real, silver_pib_nominal, silver_vab, silver_petroleo_riesgo

- Tratamiento de texto: Conversión de años provisionales (ej. "2024 (p)" → 2024).
- Estandarización Regional: Reemplazo de comas por puntos en decimales (ej. 3.450,20 → 3450.20).
- Despivoteado (Melt): Transformación del VAB provincial de formato horizontal (ancho) a vertical (largo).

Parte del código

```
"""
retropolacion_1965_2024p.xlsx | Hoja: PIB pc nominal | header=9
- Solo primeras 5 columnas
- 2024 (p) → limpiar a entero
- variacion_pct 1965 = NaN → correcto, no eliminar
"""

archivo = os.path.join(BRONZE_BCE, "retropolacion_1965_2024p.xlsx")
print(f"\n[1] {os.path.basename(archivo)}")
df = pd.read_excel(archivo, sheet_name="PIB pc nominal", engine="openpyxl", header=9)
df = df.iloc[:, :5].copy()
df.columns = ["anio", "pib_musd", "poblacion", "pib_percapita", "variacion_pct"]
df = df[df["anio"].astype(str).str.match(r"^\d{4}(\s*(p))?$", na=False)].copy()
df["anio"] = df["anio"].astype(str).str.replace(r"\s*(p)", "", regex=True).astype(int)
for col in ["pib_musd", "poblacion", "pib_percapita", "variacion_pct"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")
df = df.sort_values("anio").reset_index(drop=True)
resumen(df, "silver_pib_real")
return df
```

La función limpiar_pib_real procesa y limpia la hoja "PIB pc nominal" del archivo de Excel indicado. El código lee los datos desde la fila 9, selecciona las primeras cinco columnas y les asigna nombres estandarizados. Luego, filtra y limpia la columna de años (eliminando el sufijo (p) de los datos provisionales para convertirlos a enteros), transforma el resto de las variables a formato numérico (manejando los valores nulos correctamente), y finalmente ordena cronológicamente el set de datos antes de generar un resumen y retornar el DataFrame limpio.

inec.py — 2 funciones, 2 tablas Silver

Objetivo: Extraer las tasas de empleo y desempleo (ENEMDU) y la densidad poblacional del Censo 2022.

Tablas Generadas: silver_enemdu, silver_censo.

- Parseo de Fechas: Traducción automática de texto a fecha real (ej. "dic-07" \$ \rightarrow 2007-12-01).
- Mapeo Flexible (COLS_MAPA): Diccionario dinámico para renombrar y asegurar columnas del censo ante cambios de versión.
- Normalización: Texto de provincias y cantones formateado a "Tipo Título" (ej. "QUITO" \$ \rightarrow "Quito").

Parte del código

```
def limpiar_enemdu():
    """
    202605_Tabulados_Mercado_Laboral_EXCEL.XLSX | Hoja: '2. Tasas'
    Estructura: Encuesta | Periodo | Indicadores | Nacional | Urbana | Rural | Hombre | Mujer
    - Fila 1: encabezados principales, Fila 2: sub-encabezados -> datos desde fila 3
    - Periodo 'dic-07' -> fecha real 2007-12-01
    - Formato long: una fila por (fecha, indicador) con Nacional/Urbana/Rural
    """
    archivo = os.path.join(BRONZE_INEC, "202605_Tabulados_Mercado_Laboral_EXCEL.XLSX")
    print(f"\n[1] {os.path.basename(archivo)}")
    df = pd.read_excel(archivo, sheet_name="2. Tasas", header=None, engine="openpyxl")
    df = df.iloc[2:].copy()
    df.columns = ["encuesta", "periodo", "indicador",
                  "total_nacional", "total_urbana", "total_rural",
                  "total_hombre", "total_mujer"]
    df = df.reset_index(drop=True)
    df = df.dropna(subset=["indicador"])
    df = df[df["indicador"].astype(str).str.strip().isin(["", "nan"]) == False]

    meses = {"ene": "01", "feb": "02", "mar": "03", "abr": "04", "may": "05", "jun": "06",
             "jul": "07", "ago": "08", "sep": "09", "oct": "10", "nov": "11", "dic": "12"}

    def parsear_periodo(p):
        p = str(p).strip().lower()
        for m, n in meses.items():
            if p.startswith(m):
                y = p.split("-")[-1]
                anio = int("20"+y) if int(y) <= 30 else int("19"+y)
                return pd.Timestamp(f"{anio}-{n}-01")
        return pd.NaT

    df["periodo"] = df["periodo"].apply(parsear_periodo)
```

La función limpiar_enemdu procesa la hoja "2. Tasas" de un archivo de la ENEMDU (INEC) para estandarizar indicadores del mercado laboral. El código descarta las dos primeras filas de encabezados complejos, renombra las columnas a una estructura fija (geográfica y por género) y elimina filas sin un indicador válido. Finalmente, utiliza una función interna para estandarizar la columna de periodos de texto (como 'dic-07') convirtiéndolos en objetos de fecha reales (Timestamp), asignándoles el primer día del mes correspondiente.

supercias.py — 2 funciones, 2 tablas Silver

Parte del código

Objetivo: Cargar el ranking financiero histórico (2008-2025) y el directorio activo de empresas.

Tablas Generadas: silver_supercias_ranking, silver_supercias_directorio.

- Filtro de Relevancia: Eliminación de registros con balances en cero o vacíos.
- Validación de Identificadores: Uso de expresiones regulares para exigir RUCs estrictos de 13 dígitos.

```
def limpiar_directorio():
    """
    bi_compania.csv | sep=, | 6 columnas: expediente, ruc, nombre, tipo, pro_codigo, provi
    - ~338k empresas activas con EEFF presentados
    - pro_codigo → zfill(2) para coincidir con cod_provincia del BCE
    - RUC validado: 13 dígitos
    """
    archivo = os.path.join(BRONZE_SUPER, "bi_compania.csv")
    print(f"\n[2] {os.path.basename(archivo)}")
    df = pd.read_csv(archivo, low_memory=False)
    df["expediente"] = pd.to_numeric(df["expediente"], errors="coerce").astype("Int64")
    df["pro_codigo"] = df["pro_codigo"].astype(str).str.strip().str.zfill(2).replace("nan", "")
    df["nombre"] = df["nombre"].astype(str).str.strip().str.title()
    df["provincia"] = df["provincia"].astype(str).str.strip().str.title()
    df["tipo"] = df["tipo"].astype(str).str.strip()
    df["ruc"] = df["ruc"].astype(str).str.strip()
    n_inv = (~df["ruc"].str.match(r"^\d{13}$")).sum()
    if n_inv > 0:
        print(f" ⚠️ {n_inv} filas con RUC inválido → eliminadas")
        df = df[df["ruc"].str.match(r"^\d{13}$").copy()]
    df = df.dropna(subset=["expediente"]).reset_index(drop=True)
    resumen(df, "silver_supercias_directorio")
    print(f" Provincias únicas: {df['provincia'].nunique()}")
    return df
```

La función limpiar_directorio procesa un archivo CSV (bi_compania.csv) que contiene el registro de empresas activas de la Superintendencia de Compañías. El código estandariza el formato de los datos rellenoando con ceros a la izquierda el código de provincia (zfill(2)) para compatibilidad institucional, convierte a formato título los nombres y provincias, y valida estrictamente el RUC exigiendo que tenga exactamente 13 dígitos numéricos (eliminando los registros inválidos).

mineduc.py — 1 función, 1 tabla Silver

Parte del código

Objetivo: Analizar la infraestructura estudiantil del periodo 2023-2024.

Tablas Generadas: silver_mineduc.

- Descarte de Cabecera: Al declarar header=10, el script descarta los metadatos de las primeras filas del MINEDUC, cargando las columnas correctas de forma directa y limpia.
- Relleno de Códigos (Padding): El uso de zfill(2) y zfill(4) limpia la conversión de números a flotantes de Excel (ej: "2.0" -> "2" -> "02") asegurando compatibilidad total con códigos del INEC.
- Resiliencia de Datos: Los guiones se mapean como nulos reales para evitar errores de tipo de dato string en la inserción de enteros.

```
COLS_TEXT = ["ao_lectivo","AMIE","nombre_institucion","zona","provincia",
             "cod_provincia","canton","cod_canton","parroquia",
             "tipo_educacion","nivel_educacion","sostenimiento"]
for col in [c for c in df.columns if c not in COLS_TEXT]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df["provincia"] = df["provincia"].astype(str).str.strip().str.title()
df["canton"]    = df["canton"].astype(str).str.strip().str.title()
df["parroquia"] = df["parroquia"].astype(str).str.strip().str.title()
df["cod_provincia"] = (df["cod_provincia"].astype(str)
                      .str.split(".").str[0].str.zfill(2).replace("nan", None))
df["cod_canton"]    = (df["cod_canton"].astype(str)
                      .str.split(".").str[0].str.zfill(4).replace("nan", None))

if "bach3_femenino" in df.columns and "bach3_masculino" in df.columns:
    df["bach3_total"] = df["bach3_femenino"].fillna(0) + df["bach3_masculino"].fillna(0)
    ambos_nan = df["bach3_femenino"].isna() & df["bach3_masculino"].isna()
    df.loc[ambos_nan, "bach3_total"] = None

df = df.reset_index(drop=True)
resumen(df, "silver_mineduc")

print("\n Niveles Educación:")
print(df["nivel_educacion"].value_counts().to_string())
```

Este fragmento de código limpia y estandariza un conjunto de datos del Ministerio de Educación (Mineduc). El proceso convierte automáticamente todas las columnas que no pertenecen a las variables de texto principales (COLS_TEXT) a formato numérico, forzando los errores a nulos. Además, normaliza los nombres de ubicaciones geográficas a formato tipo título y limpia los códigos de provincia y cantón



Resultado de la capa Silver

Entre las 4 fuentes se generan 10 tablas Silver limpias en PostgreSQL, cada una con sus reglas de limpieza documentadas en el script correspondiente — listas para alimentar las vistas Gold.

PostgreSQL 18 — 14 tablas en 4 grupos



2

Dimensiones

dim_tiempo · dim_geografia



3

Tablas Fact

fact_macro_anual · fact_indicadores_diarios
· fact_empleo



5

Silver BCE

pib_real · pib_nominal · vab ·
petroleo_riesgo · iee



4

Silver INEC/Super/MINE

enemdu · censo · supercias_ranking ·
supercias_directorio · mineduc

create_tables.sql crea las 14 tablas junto con 17 índices para optimizar las consultas del dashboard.

6 vistas SQL alimentan las 3 páginas del dashboard

P1 gold_pib_tendencia PIB + clasificación de ciclo económico + variación per cápita	P1 gold_petroleo_30dias WTI + crudo ECU + promedio móvil 30 días + variación diaria	P1 6TO CICLO gold_iee_vs_pib IEE promedio anual vs variación del PIB — indicador adelantado
P2 gold_empleo_tendencia ENEMDU histórico: desempleo, subempleo, empleo adecuado	P2 6TO CICLO gold_vab_por_sector VAB por provincia: participación %, ranking, variación anual	P3 gold_bachilleres_vs_empresas MINEDUC × Supercias por provincia + ratio bachilleres/empresa



Gracias